Lecture Notes for

Neural Networks I

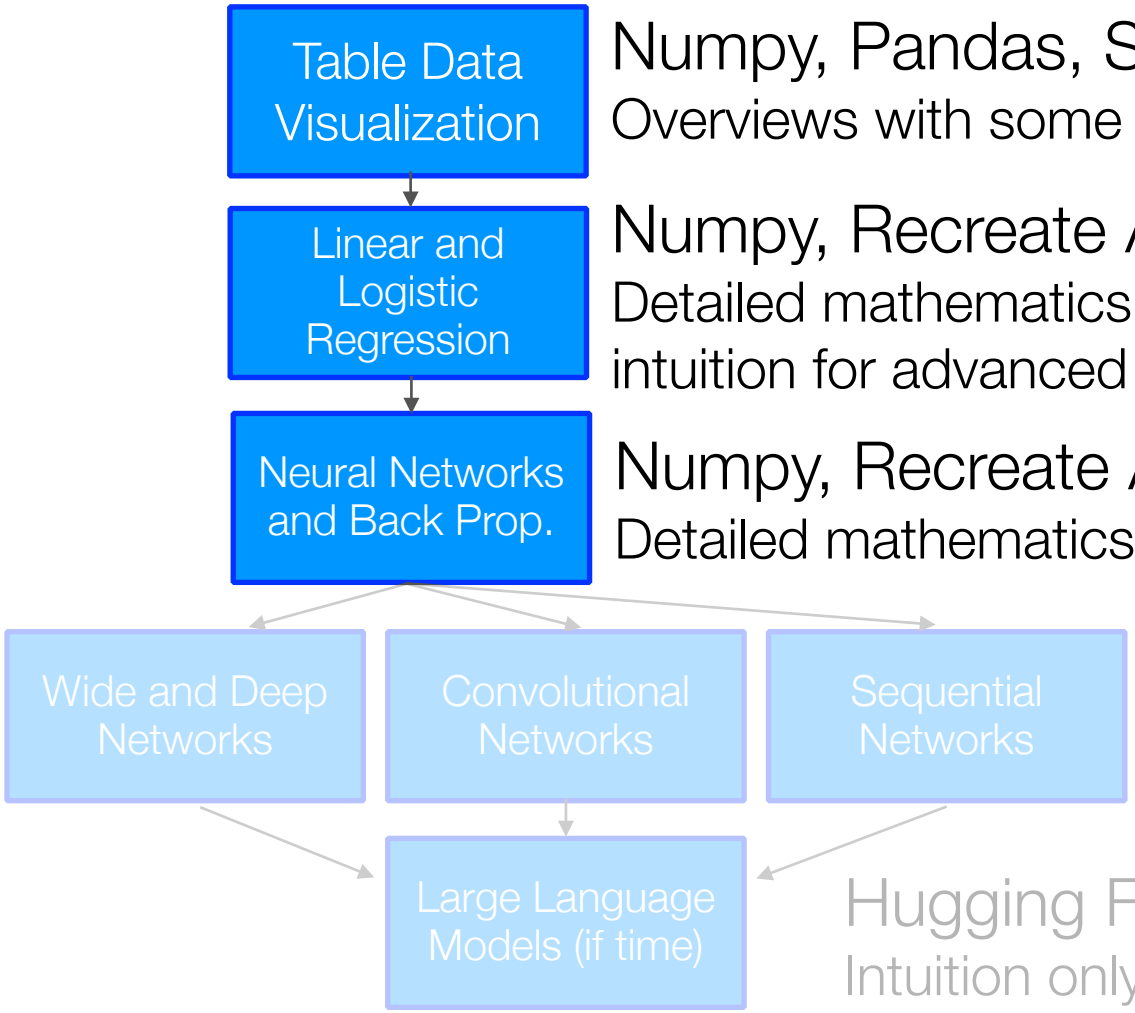


Neural Networks, a History

Class Logistics and Agenda

- Logistics:
  - Grading Update
  - **Self Test**: Finals during class versus in finals slot
  - **Next time:** Flipped Module on back propagation
- Multi Week Agenda:
  - **Today**: Neural Networks History, up to 1980
  - **Flipped**: Back Propagation Gradients for NN
  - **Next Week**: More Neural Networks x2 lecture
  - **Exam Week**: Review and Mid-term

Class Overview, by topic



Numpy, Pandas, Seaborn
Overviews with some in-depth discussion

Numpy, Recreate API for Scikit-learn
Detailed mathematics for simple optimization
intuition for advanced optimization

Numpy, Recreate API for DL Frameworks
Detailed mathematics for NN operations

Hugging Face
Intuition only

Last Time: LR with Newton's Method

- **Objective Function**: likelihood, given below
- **Parameters**: changing $\mathbf{w}$
- **Update Direction**: need to calculate gradient to know how to update initial guess of $\mathbf{w}$

$$L(\mathbf{w}) = \prod_i g\left(\mathbf{x}^{(i)} \cdot \mathbf{w}\right)_{y^{(i)}=1} \cdot \left(1 - g(\mathbf{x}^{(i)} \cdot \mathbf{w})\right)_{y^{(i)}=0}$$

- Simplify $L(\mathbf{w})$ with **logarithm**, $l(\mathbf{w})$

$$l(\mathbf{w}) = \sum_i y^{(i)} \ln\left(g(\mathbf{x}^{(i)} \cdot \mathbf{w})\right) + (1 - y^{(i)}) \ln\left(1 - g(\mathbf{x}^{(i)} \cdot \mathbf{w})\right)$$

Newton's Method, the Hessian

Function of single variable:

$$w \leftarrow w - \underbrace{[\frac{\partial^2}{\partial w} l(w)]^{-1}}_{\text{inverse 2nd deriv}} \underbrace{\frac{\partial}{\partial w} l(w)}_{\text{derivative}}$$

Function multivariate:

$$\mathbf{w} \leftarrow \mathbf{w} + \eta \cdot \underbrace{\mathbf{H}(\mathbf{w})^{-1}}_{\text{inverse Hessian}} \cdot \underbrace{\nabla l(\mathbf{w})}_{\text{gradient}}$$

$$\nabla^2 l(\mathbf{w}) = \mathbf{H}(\mathbf{w}) = \begin{bmatrix} \frac{\partial^2}{\partial w_1} l(\mathbf{w}), & \frac{\partial}{\partial w_1}\frac{\partial}{\partial w_2} l(\mathbf{w}), & \dots & \frac{\partial}{\partial w_1}\frac{\partial}{\partial w_{N-1}} l(\mathbf{w}) & \frac{\partial}{\partial w_1}\frac{\partial}{\partial w_N} l(\mathbf{w}) \\ \frac{\partial}{\partial w_2}\frac{\partial}{\partial w_1} l(\mathbf{w}), & \frac{\partial^2}{\partial w_2} l(\mathbf{w}), & \dots & \frac{\partial}{\partial w_2}\frac{\partial}{\partial w_{N-1}} l(\mathbf{w}) & \frac{\partial}{\partial w_2}\frac{\partial}{\partial w_N} l(\mathbf{w}) \\ & \vdots & \ddots & & \vdots \\ \frac{\partial}{\partial w_N}\frac{\partial}{\partial w_1} l(\mathbf{w}), & \frac{\partial}{\partial w_N}\frac{\partial}{\partial w_2} l(\mathbf{w}), & \dots & \frac{\partial}{\partial w_N}\frac{\partial}{\partial w_{N-1}} l(\mathbf{w}) & \frac{\partial^2}{\partial w_N} l(\mathbf{w}) \end{bmatrix}$$

$$\mathbf{w} \leftarrow \mathbf{w} + \eta \cdot \underbrace{\left[\mathbf{X}^T \cdot \text{diag}\left[g(\mathbf{X} \cdot \mathbf{w}) \odot (1 - g(\mathbf{X} \cdot \mathbf{w}))\right] \cdot \mathbf{X}\right]^{-1}}_{\text{inverse Hessian}} \cdot \underbrace{\mathbf{X} \odot y_{diff}}_{\text{gradient}}$$

```
MultiClass Logistic Regression Object with coefficients:
[[ -5.70420524   1.68548675   1.71728225  -9.17153356  -3.17086217]
 [ -0.86722628  -0.40201342  -1.0704752    3.58134621  -3.04194971]
 [-18.43258088  -0.15820534  -2.95982253  15.38047795  10.50512266]]
Accuracy of:  0.9666666666666667
CPU times: user 16.6 ms, sys: 4.27 ms, total: 20.9 ms
Wall time: 6.46 ms
```

Ours: Hessian, Multiclass

```
[[-6.04613316  4.18486664 -9.72680649 -8.94431158]
 [ 2.95106109 -0.43710195 -3.85815241 -1.35579337]
 [ 3.09507206 -3.74776469 13.58495889 10.30010495]]
Accuracy of:  0.9333333333333333
CPU times: user 15.9 ms, sys: 3.01 ms, total: 18.9 ms
Wall time: 5.77 ms
```

sklearn, Multiclass (fortran)



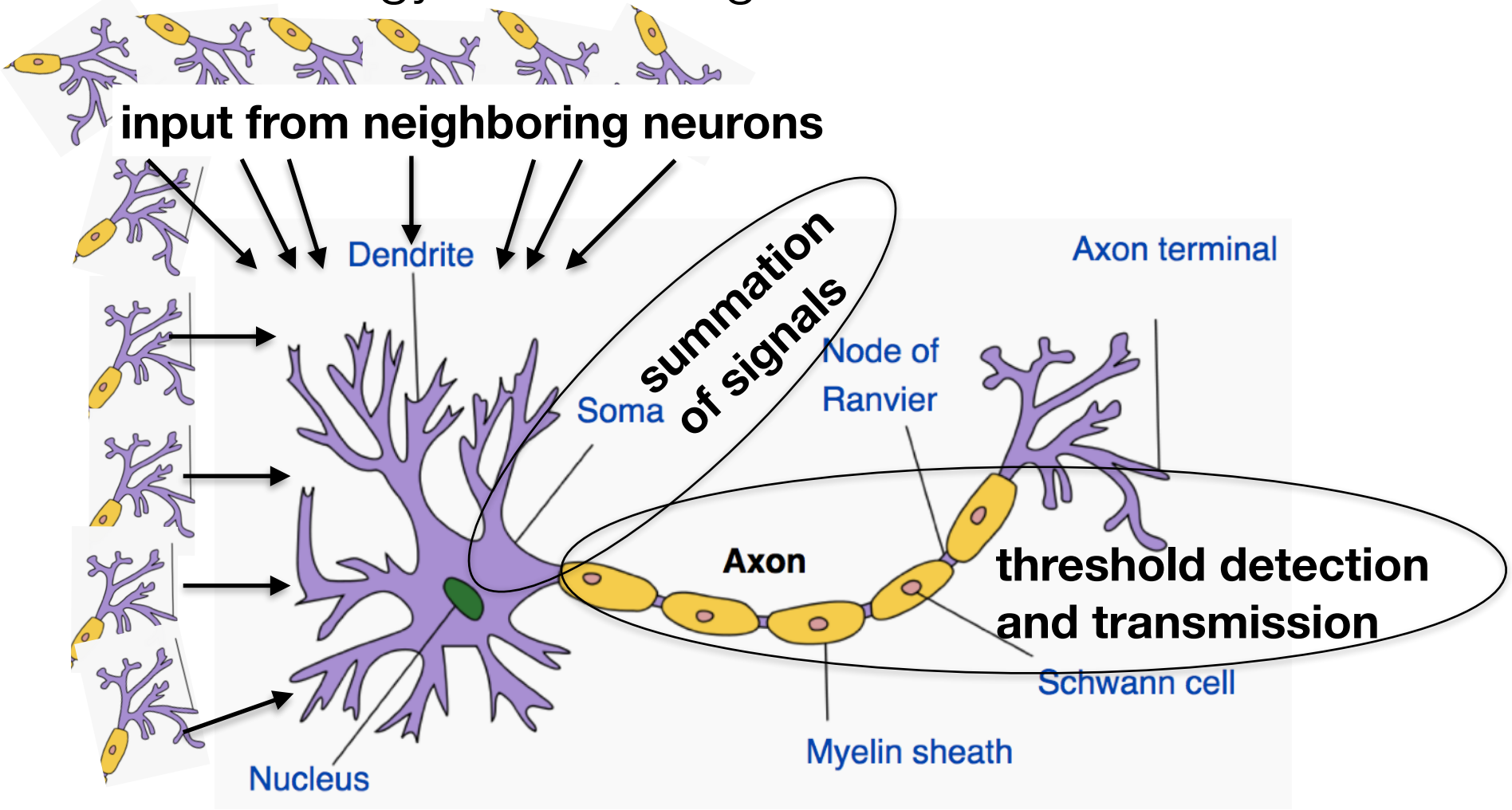
Lab 2, Town Hall (if needed)

A History of Neural Networks

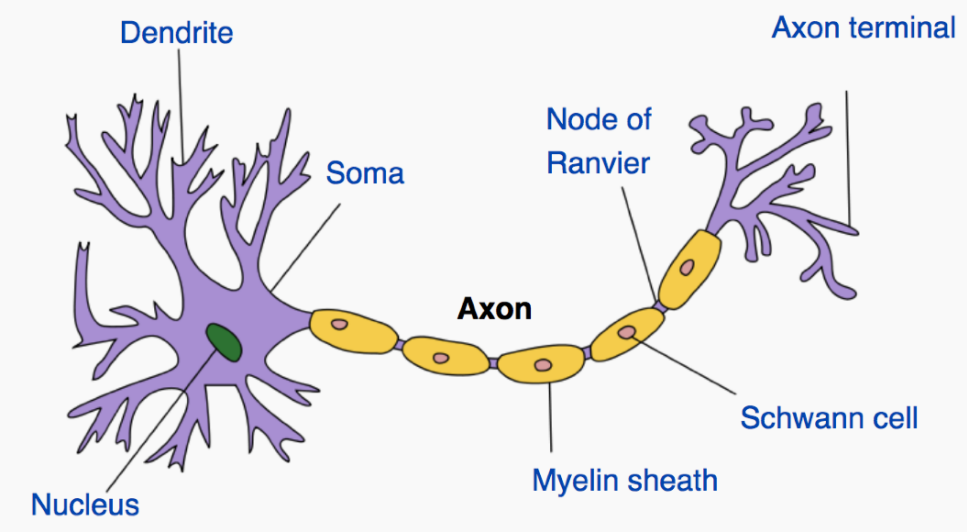


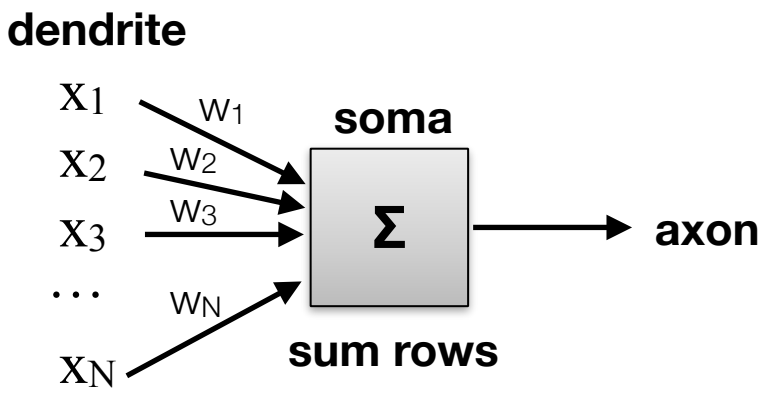
Neurons

- From biology to modeling:



McCulloch and Pitts, 1943





$$\mathbf{X} \cdot \mathbf{W} = a$$

for each neuron

logic gates of the mind



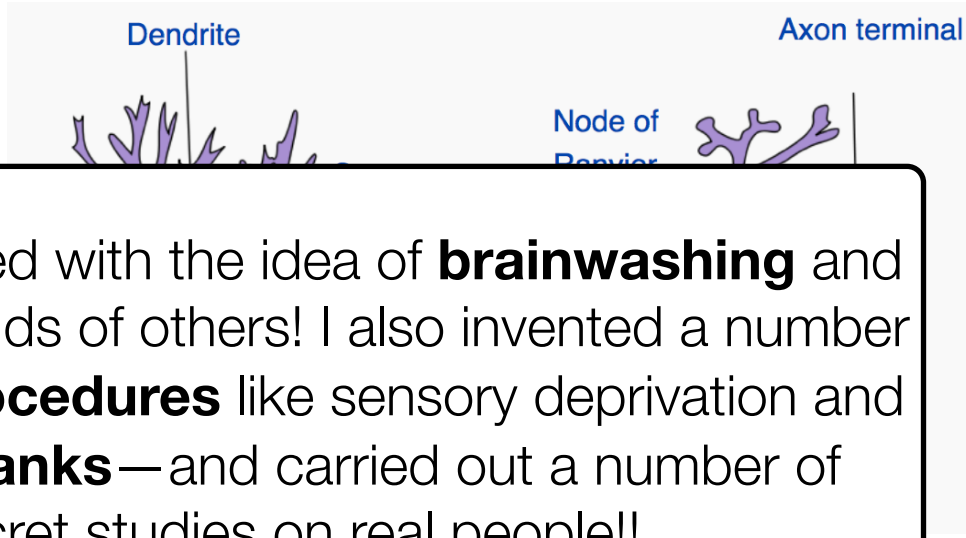
Warren McCulloch



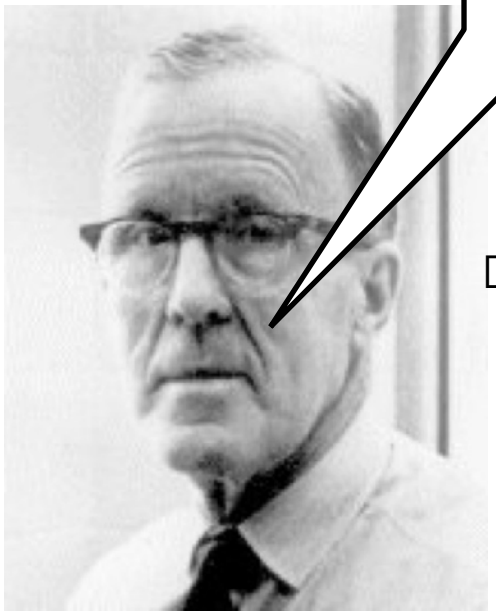
Walter Pitts

Neurons

- McCulloch and Pitts, 1943
- Donald Hebb, 1949
  - Hebb's Law: close neurons fire together
  - neurons "learn
  - easier synaptic
  - basis of neura



I was infatuated with the idea of **brainwashing** and controlling minds of others! I also invented a number of **torture procedures** like sensory deprivation and **isolation tanks**—and carried out a number of secret studies on real people!!



Donald O. Hebb



Warren McCulloch

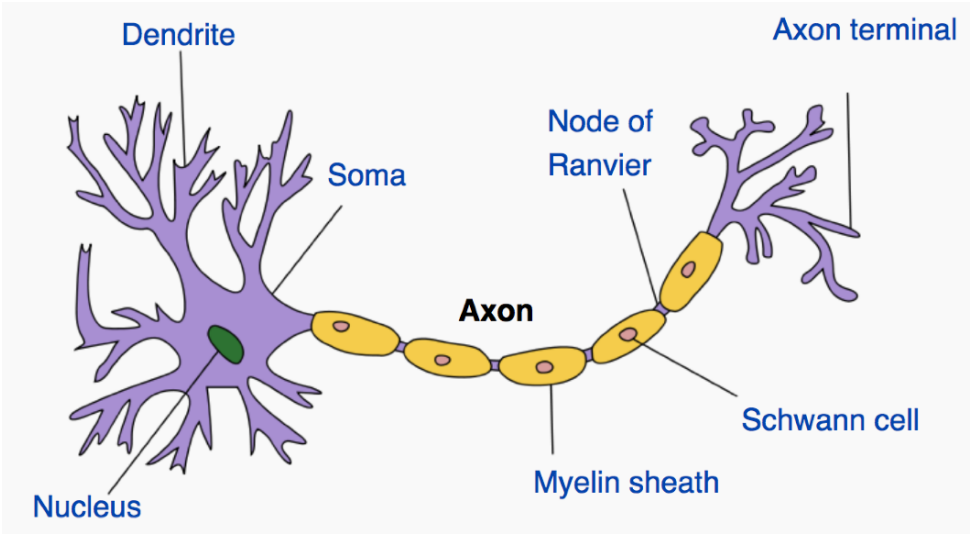


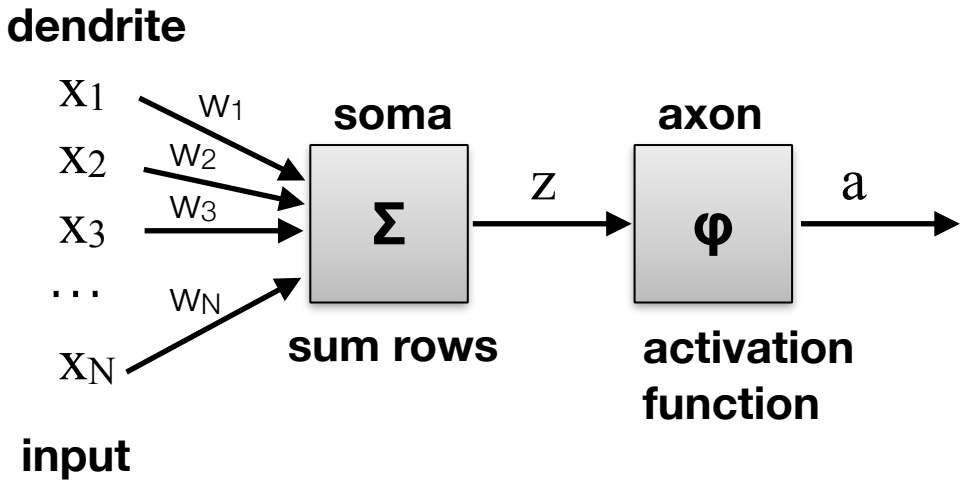
Walter Pitts

Rosenblatt's perceptron, 1957



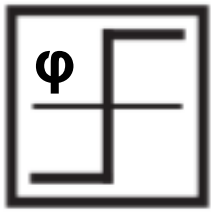
Frank Rosenblatt





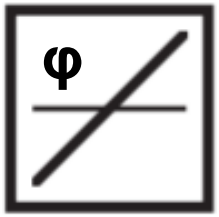
Axon Functions

hard limit



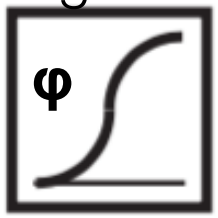
a=−1 z< 0
a= 1 z>=0

linear



a=z

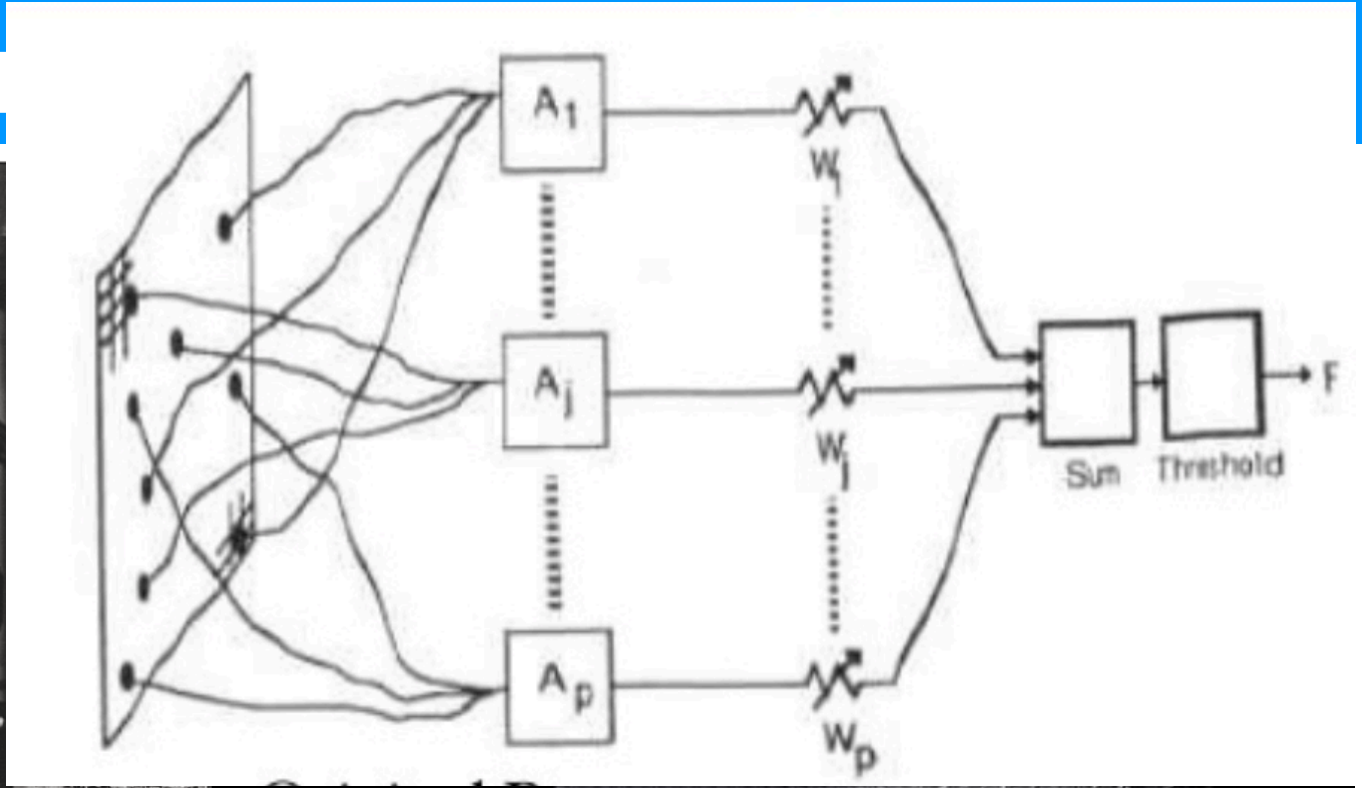
sigmoid

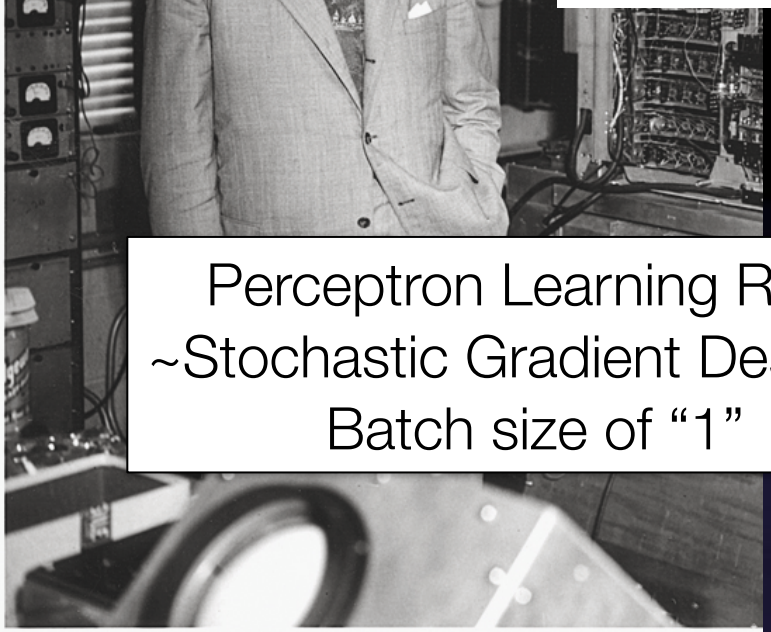


$$a=\frac{1}{1+\exp(-z)}$$

The Mark 1 P





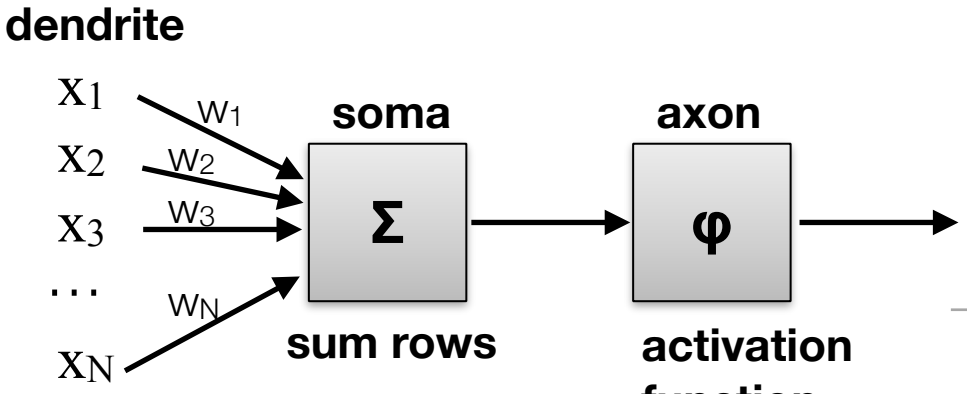


Perceptron Learning Rule:
~Stochastic Gradient Descent,
Batch size of “1”



Layers Notation for Table Data

One Neuron



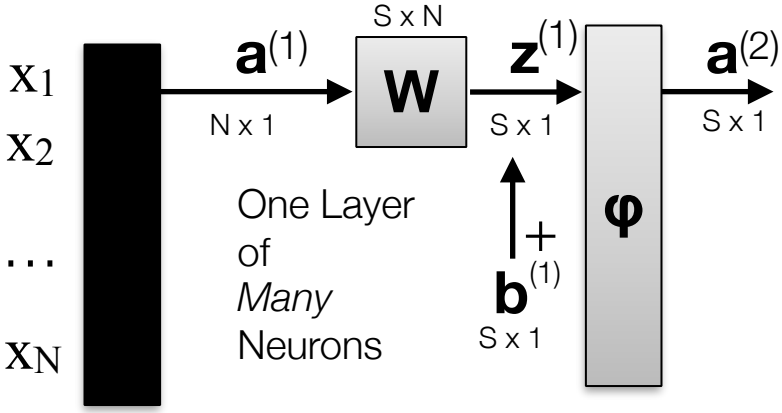
$$[\mathbf{x}^{(i)}]^T = \begin{bmatrix} x_1 \\ \vdots \\ x_j \\ \vdots \\ x_N \end{bmatrix}^{(i)} = [\mathbf{a}^{(1)}]^{(i)}$$

$$\mathbf{a} = \mathbf{x}^T$$

$$\mathbf{z} = \mathbf{W} \cdot [\mathbf{x}^{(i)}]^T + \mathbf{b}$$

one neuron →

$$\mathbf{W} = \begin{bmatrix} w_{1,1} & \cdots & w_{1,N} \\ w_{2,1} & \cdots & w_{2,N} \\ \vdots & & \\ w_{S,1} & \cdots & w_{S,N} \end{bmatrix} \quad \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_S \end{bmatrix}$$



$$[\mathbf{z}^{(1)}]^{(i)} = \mathbf{W}^{(1)} \cdot [\mathbf{a}^{(1)}]^{(i)} + \mathbf{b}^{(1)}$$

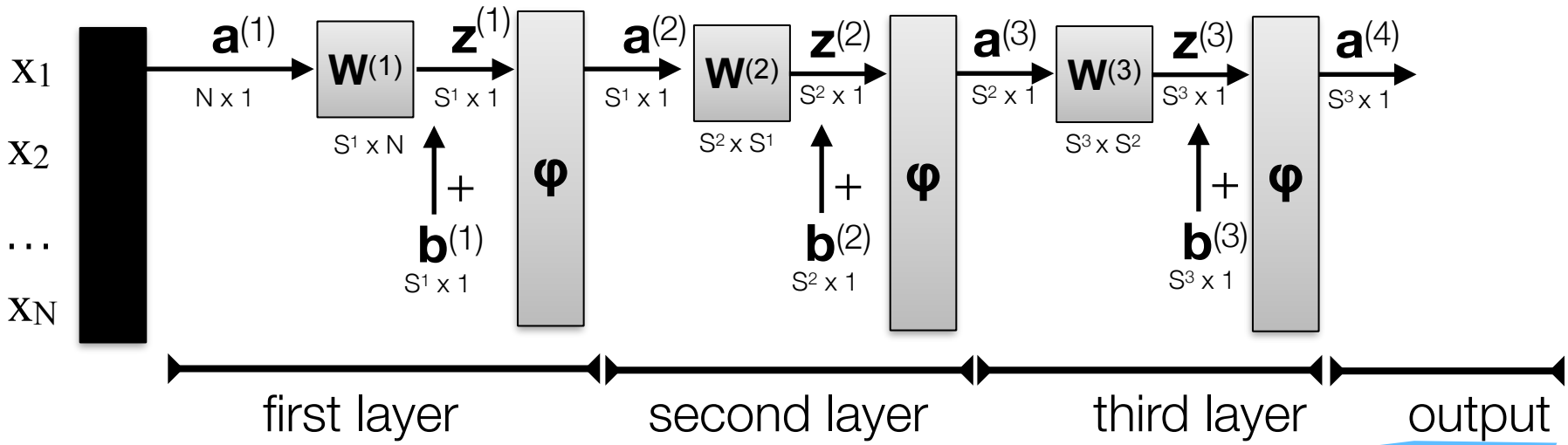
$$\mathbf{a}^{(L)} = \phi(\mathbf{z}^{(L-1)}) \qquad [\mathbf{a}^{(L)}]^{(i)} = \phi\left([\mathbf{z}^{(L-1)}]^{(i)}\right)$$

$$\mathbf{a}^{(next)} = \begin{bmatrix} \phi(z_1^{curr}) \\ \vdots \\ \phi(z_N^{curr}) \end{bmatrix} \rightarrow \mathbf{a}^{(L)} = \begin{bmatrix} \phi(z_1^{L-1}) \\ \vdots \\ \phi(z_N^{L-1}) \end{bmatrix}$$

$\mathbf{x}^{(i)}$ One ***row*** from Table data as a input ***column*** to model

notation adapted from *Neural Network Design*, Hagan, Demuth, Beale, and De Jesus

Shorthand Multiple Layers Notation



$$\mathbf{a}^{(L+1)} = \phi(\mathbf{z}^{(L)})$$

$\mathbf{a}^{(final)}$ size=unique classes, C

$$\mathbf{z}^{(L)} = \mathbf{W}^{(L)} \cdot \mathbf{a}^{(L)} + \mathbf{b}^{(L)} \quad \longleftrightarrow \quad [\mathbf{z}^{(L)}]^{(i)} = \mathbf{W}^{(L)} \cdot [\mathbf{a}^{(L)}]^{(i)} + \mathbf{b}^{(L)}$$

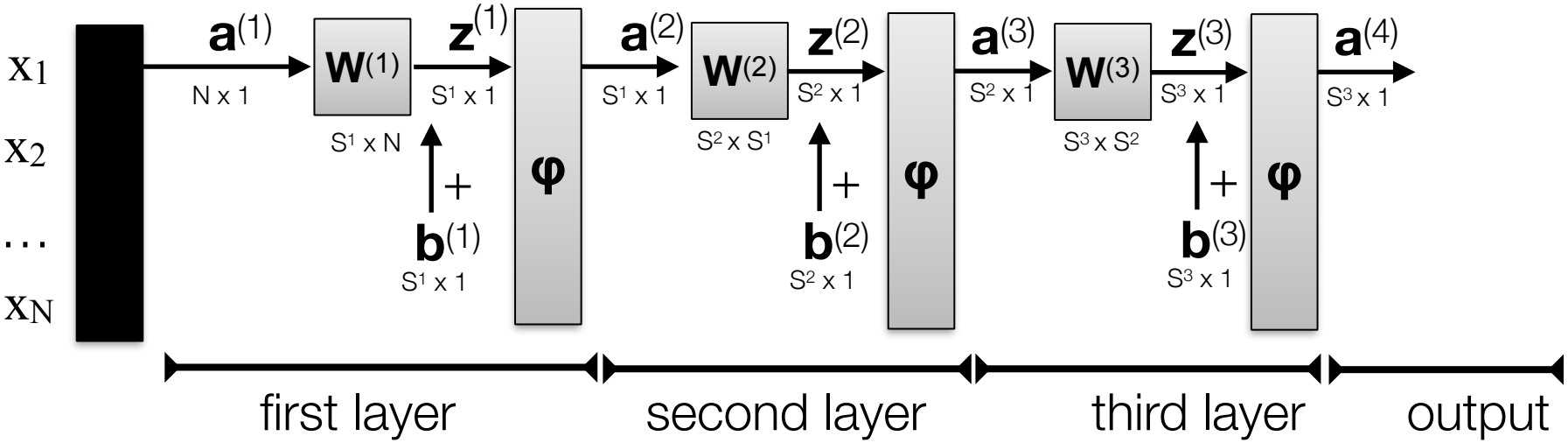
often we just use shorthand

$$\mathbf{z}^{(L)} = \mathbf{W}^{(L)} \cdot \phi(\mathbf{z}^{(L-1)}) + \mathbf{b}^{(L)}$$

$$\mathbf{z}^{(L)} = \mathbf{W}^{(L)} \cdot \phi\left(\mathbf{W}^{(L-1)} \cdot \phi(\mathbf{z}^{(L-2)}) + \mathbf{b}^{(L-1)}\right) + \mathbf{b}^{(L)}$$

notation adapted from *Neural Network Design*, Hagan, Demuth, Beale, and De Jesus

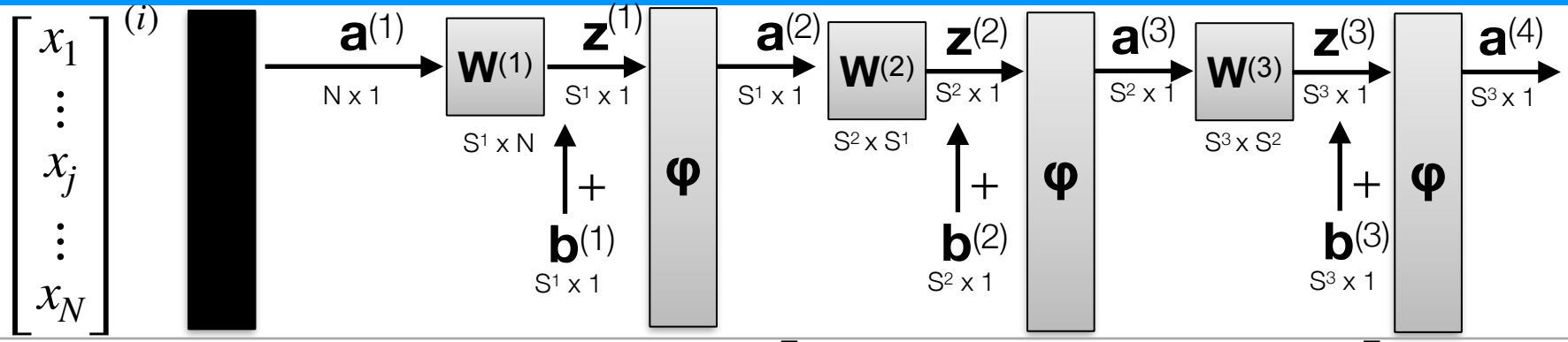
Parameters and Training



- **Self test**: How many parameters need to be trained in the above network?
  - A. $[(N+1) \times S^1] + [(S^1+1) \times S^2] + [(S^2+1) \times S^3]$
  - B. $|\mathbf{W}^{(1)}| + |\mathbf{W}^{(2)}| + |\mathbf{W}^{(3)}| + |\mathbf{b}^{(1)}| + |\mathbf{b}^{(2)}| + |\mathbf{b}^{(3)}|$
  - C. can't determine from diagram
  - D. it depends on the sizes of intermediate variables, $\mathbf{z}^{(i)}$

notation adapted from *Neural Network Design*, Hagan, Demuth, Beale, and De Jesus

Compact feedforward notation



$$\mathbf{X}^T = \left[\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix}^{(1)}, \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix}^{(2)} \cdots \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix}^{(M)} \right] = \left[\begin{bmatrix} a_0^{(1)} \\ a_1^{(1)} \\ \vdots \\ a_N^{(1)} \end{bmatrix}^{(1)}, \begin{bmatrix} a_0^{(1)} \\ a_1^{(1)} \\ \vdots \\ a_N^{(1)} \end{bmatrix}^{(2)} \cdots \begin{bmatrix} a_0^{(1)} \\ a_1^{(1)} \\ \vdots \\ a_N^{(1)} \end{bmatrix}^{(M)} \right] = \mathbf{A}^{(1)}$$

Table Data

Table Data, using $\mathbf{A}$ Notation

L^{th} layer intermediate variables

$$\mathbf{A}^{(L)} = \left[\begin{bmatrix} a_0^{(L)} \\ a_1^{(L)} \\ \vdots \\ a_N^{(L)} \end{bmatrix}^{(1)}, \begin{bmatrix} a_0^{(L)} \\ a_1^{(L)} \\ \vdots \\ a_N^{(L)} \end{bmatrix}^{(2)} \cdots \begin{bmatrix} a_0^{(L)} \\ a_1^{(L)} \\ \vdots \\ a_N^{(L)} \end{bmatrix}^{(M)} \right]$$

$$\mathbf{Z}^{(L)} = \left[\begin{bmatrix} z_0^{(L)} \\ z_1^{(L)} \\ \vdots \\ z_N^{(L)} \end{bmatrix}^{(1)}, \begin{bmatrix} z_0^{(L)} \\ z_1^{(L)} \\ \vdots \\ z_N^{(L)} \end{bmatrix}^{(2)} \cdots \begin{bmatrix} z_0^{(L)} \\ z_1^{(L)} \\ \vdots \\ z_N^{(L)} \end{bmatrix}^{(M)} \right]$$

Compact feedforward notation

$$\mathbf{Z}^{(L)} = \mathbf{W}^{(L)} \cdot \mathbf{A}^{(L)} + \mathbf{b}^{(L)}$$

$$\mathbf{Z}^{(L)} = \mathbf{W}^{(L)} \cdot \phi(\mathbf{Z}^{(L-1)}) + \mathbf{b}^{(L)}$$

$$\mathbf{z}^{(L)} = \mathbf{W}^{(L)} \cdot \mathbf{a}^{(L)} + \mathbf{b}^{(L)}$$

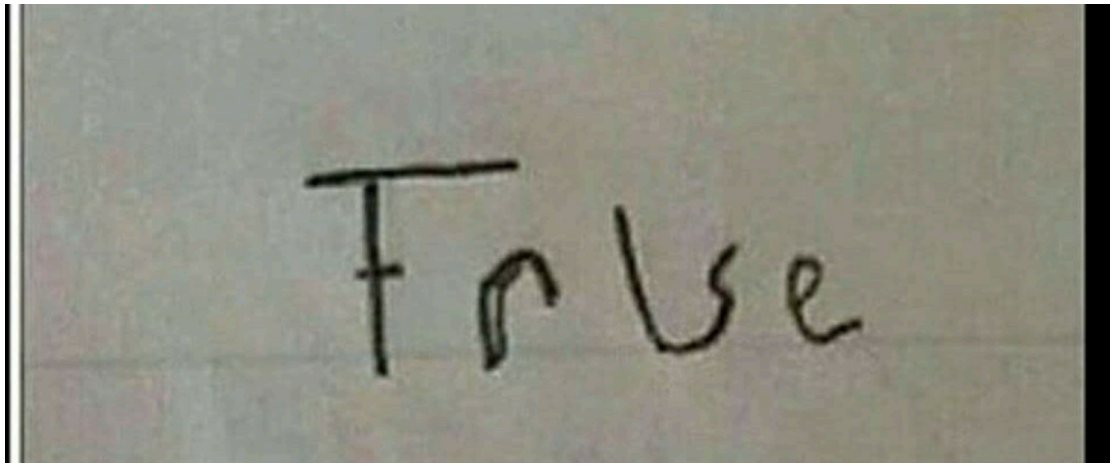
$$\left[\mathbf{z}^{(L)}\right]^{(i)} = \mathbf{W}^{(L)} \cdot \left[\mathbf{a}^{(L)}\right]^{(i)} + \mathbf{b}^{(L)}$$

$$\begin{bmatrix} z_1^{(L)} \\ z_2^{(L)} \\ \vdots \\ z_{S^L}^{(L)} \end{bmatrix}^{(i)} = \mathbf{W}^{(L)} \cdot \begin{bmatrix} a_0^{(L)} \\ a_1^{(L)} \\ \vdots \\ a_{S^{L-1}}^{(L)} \end{bmatrix}^{(i)} + \mathbf{b}^{(L)}$$

$$\left[\begin{bmatrix} z_1^{(L)} \\ z_2^{(L)} \\ \vdots \\ z_{S^L}^{(L)} \end{bmatrix}^{(1)}, \begin{bmatrix} z_1^{(L)} \\ z_2^{(L)} \\ \vdots \\ z_{S^L}^{(L)} \end{bmatrix}^{(2)} \cdots \begin{bmatrix} z_1^{(L)} \\ z_2^{(L)} \\ \vdots \\ z_{S^L}^{(L)} \end{bmatrix}^{(M)} \right] = \mathbf{W}^{(L)} \cdot \left[\begin{bmatrix} a_0^{(L)} \\ a_1^{(L)} \\ \vdots \\ a_{S^{L-1}}^{(L)} \end{bmatrix}^{(1)}, \begin{bmatrix} a_0^{(L)} \\ a_1^{(L)} \\ \vdots \\ a_{S^{L-1}}^{(L)} \end{bmatrix}^{(2)} \cdots \begin{bmatrix} a_0^{(L)} \\ a_1^{(L)} \\ \vdots \\ a_{S^{L-1}}^{(L)} \end{bmatrix}^{(M)} \right] + \mathbf{b}^{(L)}$$

$\mathbf{b}$ is broadcast added

Historical Training of Neural Network Architectures

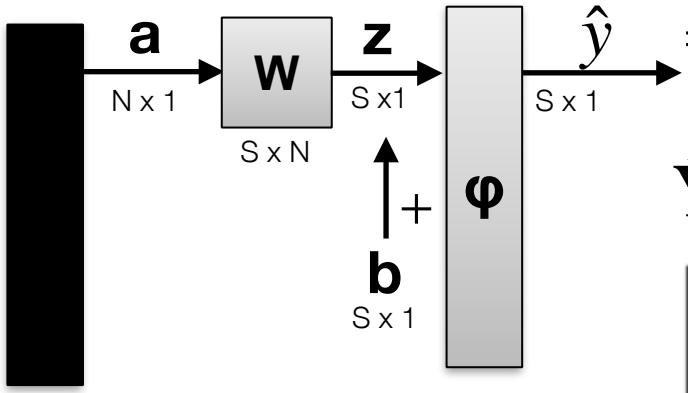


When a binary classification model outputs 0.5



One Layer Linear Architectures

- Adaline network, Widrow and Hoff, 1960



==scalar, continuous output

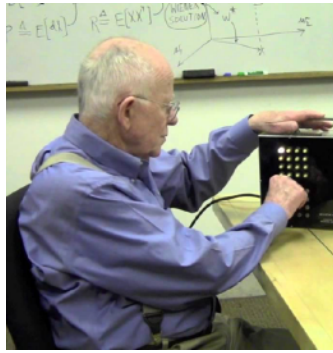
$$\hat{\mathbf{Y}} = \phi(\mathbf{w} \cdot \mathbf{A}) = \mathbf{w} \cdot \mathbf{A}$$



linear



Marcian "Ted" Hoff



Bernard Widrow

Simplify Objective Function:

$$J(\mathbf{W}) = \left\| \mathbf{Y} - \hat{\mathbf{Y}} \right\|^2 \longrightarrow J(\mathbf{w}) = \left\| \mathbf{Y} - \mathbf{w} \cdot \mathbf{A} \right\|^2$$

Need gradient $\nabla J(\mathbf{w})$ for use in update equation $\mathbf{w} \leftarrow \mathbf{w} + \eta \nabla J(\mathbf{w})$

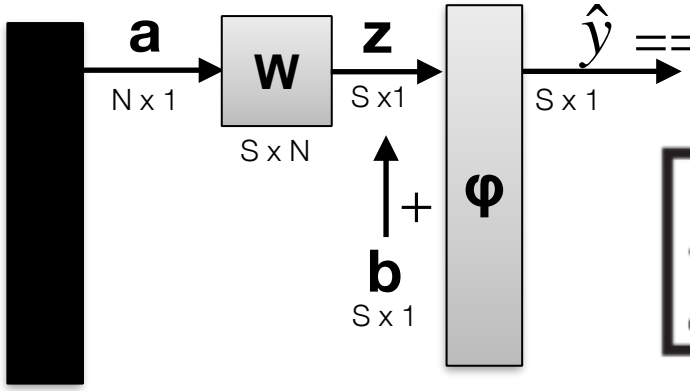
Note: We have been using the *Widrow-Hoff Learning Rule*

One Layer Linear Architectures



Marcian "Ted" Hoff

- Adaline network, Widrow and Hoff, 1960

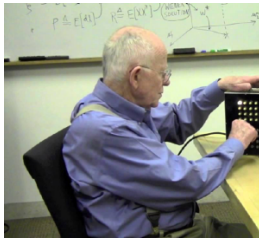


$\hat{y}$ == scalar, continuous output

$$\hat{\mathbf{Y}} = \mathbf{w} \cdot \mathbf{A}$$



linear



Bernard Widrow

Need gradient $\nabla J(\mathbf{w})$ for update equation $\mathbf{w} \leftarrow \mathbf{w} + \eta \nabla J(\mathbf{w})$

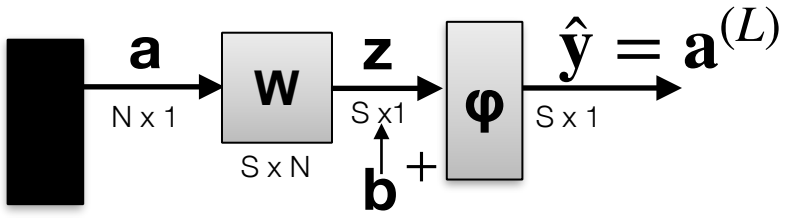
For case S=1, $\mathbf{W}$ has only one row, $\mathbf{w}$
this is just **linear regression…**

$$J(\mathbf{w}) = \sum_{i=1}^{M} (y^{(i)} - \mathbf{w} \cdot \mathbf{x}^{(i)})^2$$

$$\mathbf{w} = (\mathbf{X}^T \cdot \mathbf{X})^{-1} \cdot \mathbf{X}^T \cdot y$$



From Regression to Classification



No longer regression, $\mathbf{Y}$ is a category

ground truth $\mathbf{Y}$ is one-hot encoded!

Ground Truth Labels

$$\mathbf{Y} = \left[\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_C \end{bmatrix}^{(1)} \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_C \end{bmatrix}^{(2)} \cdots \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_C \end{bmatrix}^{(M)} \right] \rightarrow \left[\begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}^{(1)} \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 1 \end{bmatrix}^{(2)} \cdots \begin{bmatrix} 0 \\ 1 \\ \vdots \\ 0 \end{bmatrix}^{(M)} \right]$$

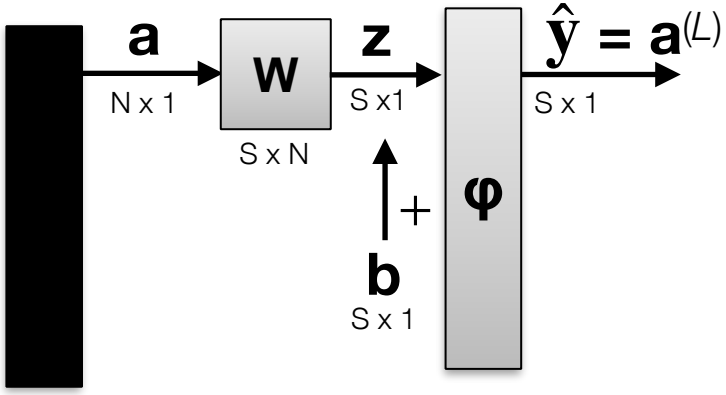
Need objective Function, minimize MSE $J(\mathbf{W}) = \| \mathbf{Y} - \hat{\mathbf{Y}} \|^2$

$$J(\mathbf{W}) = \left\| \underbrace{\left[\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_C \end{bmatrix}^{(1)} \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_C \end{bmatrix}^{(2)} \cdots \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_C \end{bmatrix}^{(M)} \right]}_{\mathbf{Y}} - \underbrace{\left[\begin{bmatrix} a_1^{(L)} \\ a_2^{(L)} \\ \vdots \\ a_C^{(L)} \end{bmatrix}^{(1)} \begin{bmatrix} a_1^{(L)} \\ a_2^{(L)} \\ \vdots \\ a_C^{(L)} \end{bmatrix}^{(2)} \cdots \begin{bmatrix} a_1^{(L)} \\ a_2^{(L)} \\ \vdots \\ a_C^{(L)} \end{bmatrix}^{(M)} \right]}_{\hat{\mathbf{Y}}} \right\|^2$$

One Layer Classification



- Rosenblatt's perceptron, 1957



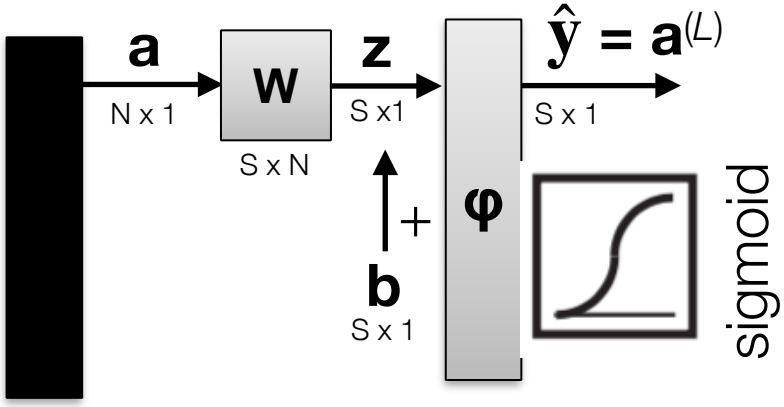
Self Test - If this is a binary classification problem, how large is *S*, the length of $\hat{\mathbf{y}}$ and number of rows in **W**?

A. Can't determine
B. 2
C. 1
D. N

One Layer Classification



- Modern Perceptron network



$$g(z) = \phi(z) = \frac{1}{1 + \exp(-z)}$$

$$\phi(\mathbf{w} \cdot \mathbf{x} + \mathbf{b}) = \frac{1}{1 + \exp(-\mathbf{w} \cdot \mathbf{x} + \mathbf{b})}$$

Need gradient $\nabla J(\mathbf{w})$ for update equation $\mathbf{w} \leftarrow \mathbf{w} + \eta \nabla J(\mathbf{w})$

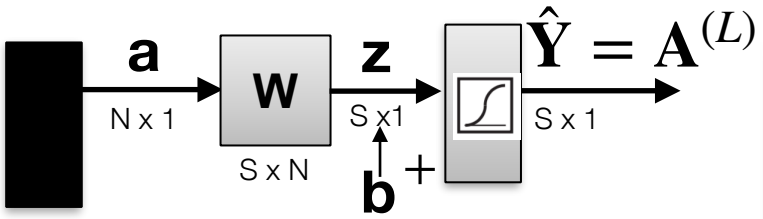
For case S=1, this is just **logistic regression…** and **we have already solved this**!

$$\mathbf{w} \leftarrow \mathbf{w} + \eta \cdot \text{mean}\left((\mathbf{y} - g(\mathbf{X} \cdot \mathbf{w})) \odot \mathbf{X}\right)_{cols}$$



What happens when S > 1 ?

One Layer Architectures of Many Classes



$$J(\mathbf{W}) = \left\| \mathbf{Y} - \hat{\mathbf{Y}} \right\|^2$$

$$J(\mathbf{W}) = \left\| \mathbf{Y} - \phi\left(\mathbf{W} \cdot \mathbf{X}^T\right) \right\|^2$$

$$J(\mathbf{w}_{row=1}) = \sum_i \left(y_1^{(i)} - \phi\left(\mathbf{w}_{row=1} \cdot \mathbf{x}^{(i)} + b_1\right) \right)^2$$

… for each class/row …

$$J(\mathbf{w}_{row=C}) = \sum_i \left(y_C^{(i)} - \phi\left(\mathbf{w}_{row=C} \cdot \mathbf{x}^{(i)} + b_C\right) \right)^2$$

$$\mathbf{Y} = \left[\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_C \end{bmatrix}^{(1)} \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_C \end{bmatrix}^{(2)} \cdots \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_C \end{bmatrix}^{(M)} \right] \rightarrow \left[\begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}^{(1)} \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 1 \end{bmatrix}^{(2)} \cdots \begin{bmatrix} 0 \\ 1 \\ \vdots \\ 0 \end{bmatrix}^{(M)} \right]$$

Each target class and row of $\mathbf{W}$ can be independently optimized

$$\hat{\mathbf{Y}} = \left[\begin{bmatrix} \phi(\mathbf{w}_{row=1} \cdot \mathbf{x}^{(1)} + b_1) \\ \phi(\mathbf{w}_{row=2} \cdot \mathbf{x}^{(1)} + b_2) \\ \vdots \\ \phi(\mathbf{w}_{row=C} \cdot \mathbf{x}^{(1)} + b_C) \end{bmatrix}^{(1)}, \begin{bmatrix} \phi(\mathbf{w}_{row=1} \cdot \mathbf{x}^{(2)} + b_1) \\ \phi(\mathbf{w}_{row=2} \cdot \mathbf{x}^{(2)} + b_2) \\ \vdots \\ \phi(\mathbf{w}_{row=C} \cdot \mathbf{x}^{(2)} + b_C) \end{bmatrix}^{(2)}, \cdots, \begin{bmatrix} \phi(\mathbf{w}_{row=1} \cdot \mathbf{x}^{(M)} + b_1) \\ \phi(\mathbf{w}_{row=2} \cdot \mathbf{x}^{(M)} + b_2) \\ \vdots \\ \phi(\mathbf{w}_{row=C} \cdot \mathbf{x}^{(M)} + b_C) \end{bmatrix}^{(M)} \right]$$



which is one versus-all!

Single Layer Early Architectures: Summary

- Adaline network, Widrow and Hoff, 1960
  - linear regression, iterative updates
- Perceptron
  - NN *with sigmoid*: logistic regression
- Multi-class perceptron: one versus all





- **But what about when we have more than one layer?**

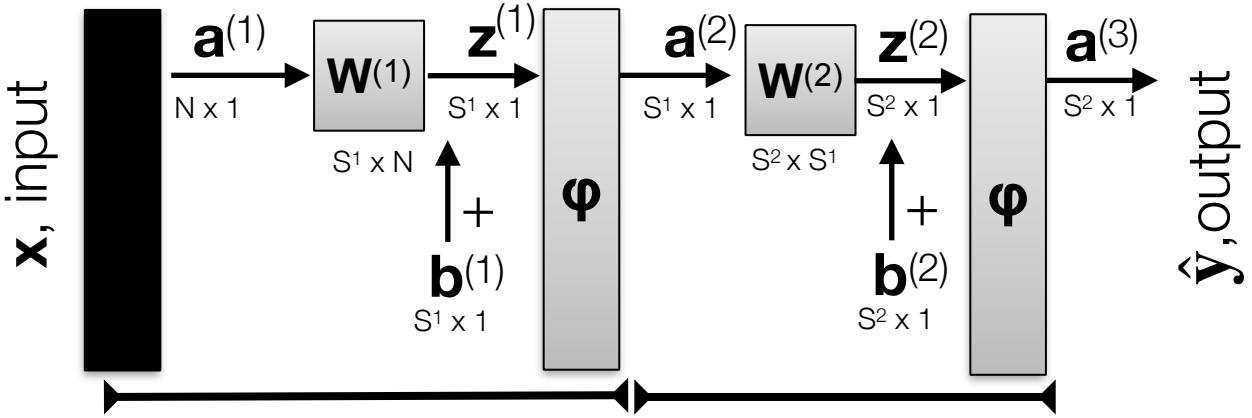




Beyond Single Layer Networks

Moving to multiple layers…

- The multi-layer perceptron (MLP):
  - two layers shown, but could be arbitrarily many layers



each element of $\hat{\mathbf{y}}$ is no longer independent.

$\mathbf{W}^{(1)}$ used for all classes so we cannot optimize using one versus all 😢

$$\begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_C \end{bmatrix}^{(i)} = \begin{bmatrix} \phi\left(\phi([\mathbf{z}^{(1)}]^{(i)}) \cdot \mathbf{w}^{(2)}_{row=1} + b^{(2)}_1\right) \\ \phi\left(\phi([\mathbf{z}^{(1)}]^{(i)}) \cdot \mathbf{w}^{(2)}_{row=2} + b^{(2)}_2\right) \\ \vdots \\ \phi\left(\phi([\mathbf{z}^{(1)}]^{(i)}) \cdot \mathbf{w}^{(2)}_{row=C} + b^{(2)}_c\right) \end{bmatrix}$$

$$[\mathbf{z}^{(1)}]^{(i)} = \mathbf{W}^{(1)} \cdot [\mathbf{a}^{(1)}]^{(i)} + \mathbf{b}^{(1)}$$

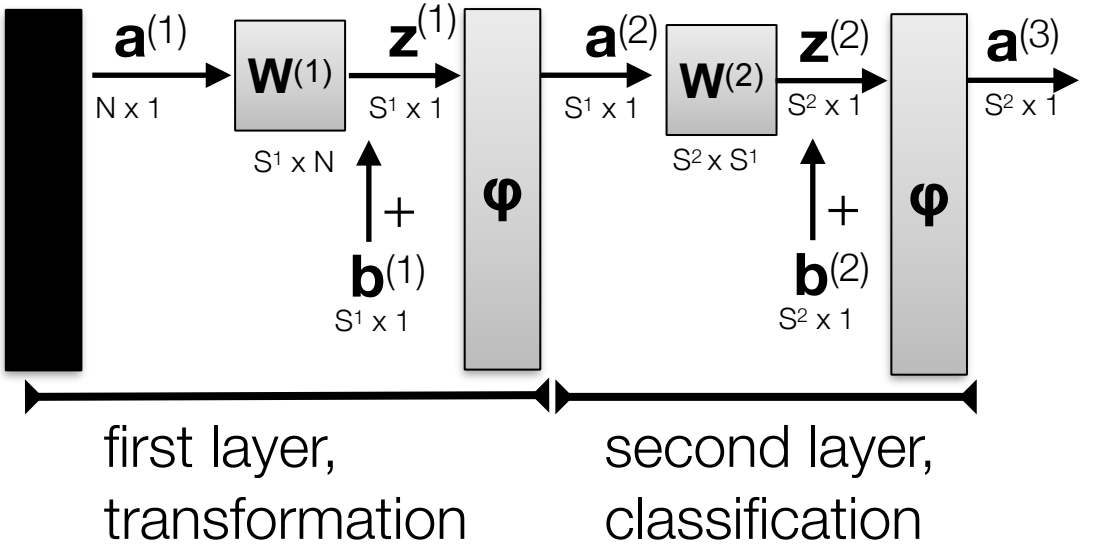


Back propagation

- Optimize all weights of network at once, using chain rule many times…
  1. Forward propagate to get all $\mathbf{Z}^{(l)}, \mathbf{A}^{(l)}$
  2. Get final layer gradient, $\partial J(\cdot)/\partial \mathbf{a}^{(L+1)}$
  3. Back propagate sensitivities (chain rule, $\partial \mathbf{z}/\partial \mathbf{a}$)
  4. Update each variable (chain rule, $\partial \mathbf{z}/\partial \mathbf{W}$ and $\partial \mathbf{z}/\partial \mathbf{b}$)



solved in flipped assignment!!



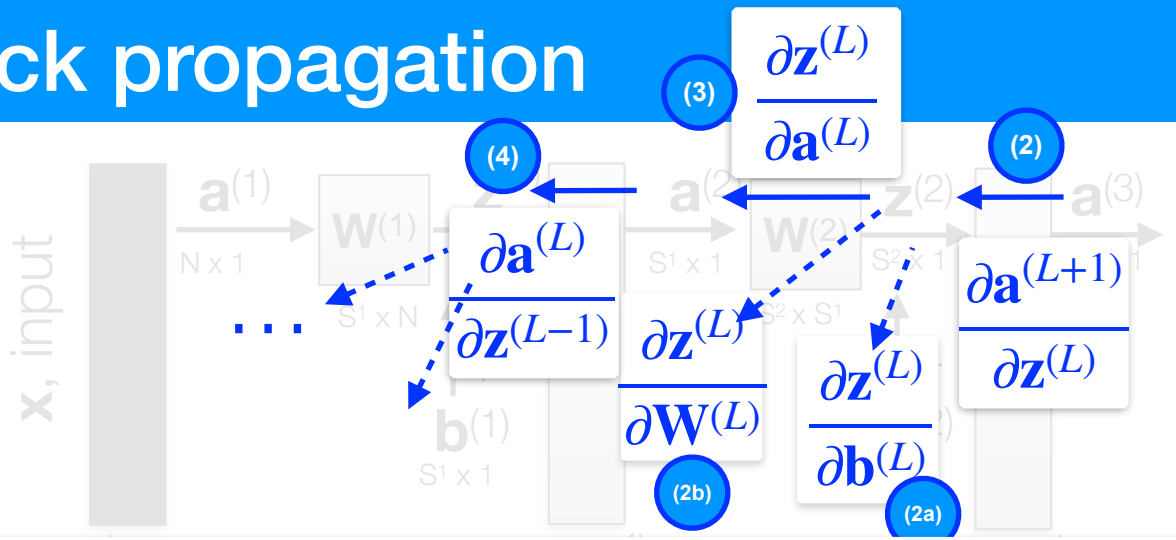
Objective Function $J(\mathbf{W}) = \left\| \mathbf{Y} - \hat{\mathbf{Y}} \right\|^2$

Parameter Updates

$$w_{i,j}^{(l)} \leftarrow w_{i,j}^{(l)} - \eta \frac{\partial J(\mathbf{W})}{\partial w_{i,j}^{(l)}}$$

$$b_i^{(l)} \leftarrow b_i^{(l)} - \eta \frac{\partial J(\mathbf{W})}{\partial b_i^{(l)}}$$

Back propagation



$$J(\mathbf{W}) = \left\| \mathbf{Y} - \hat{\mathbf{Y}} \right\|^2$$

$$w_{i,j}^{(l)} \leftarrow w_{i,j}^{(l)} - \eta \frac{\partial J(\mathbf{W})}{\partial w_{i,j}^{(l)}}$$

Each layer needs $\partial \mathbf{z}/\partial \mathbf{a}$ for back propagating to previous layer
often called the **sensitivity**
and needs $\partial \mathbf{z}/\partial \mathbf{W}$ and $\partial \mathbf{z}/\partial \mathbf{b}$ for updating its parameters

$$\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \eta \frac{\partial J(\cdot)}{\partial \mathbf{W}^{(l)}}$$

$$\frac{\partial J(\cdot)}{\partial \mathbf{W}^{(L)}} = \frac{\partial \mathbf{z}^{(L)}}{\partial \mathbf{W}^{(L)}} \cdot \frac{\partial \mathbf{a}^{(L+1)}}{\partial \mathbf{z}^{(L)}} \cdot \frac{\partial J(\cdot)}{\partial \mathbf{a}^{(L+1)}}$$

(2b) (2) (1)

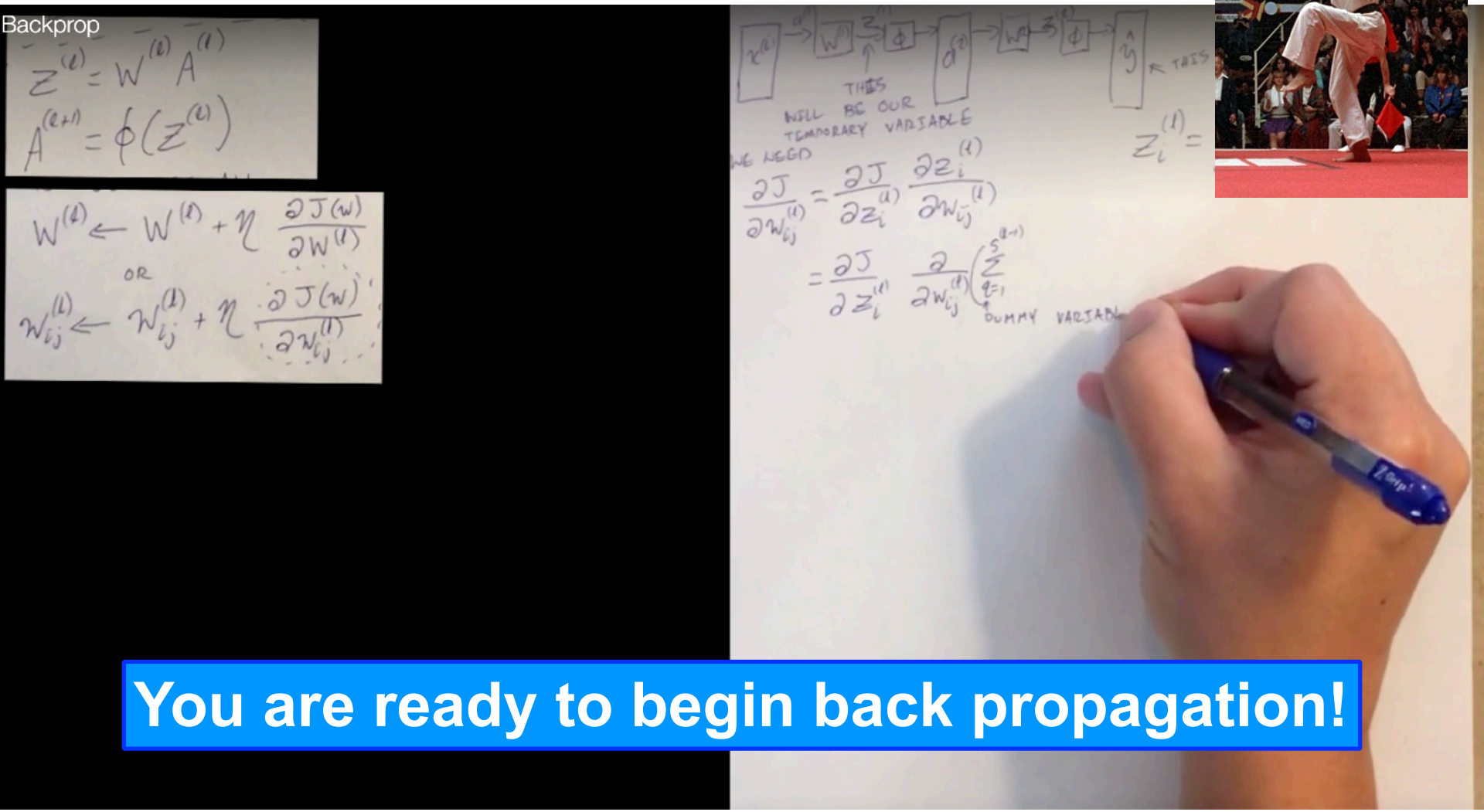
chain for parameter updates

$$\frac{\partial J(\cdot)}{\partial \mathbf{W}^{(L-1)}} = \frac{\partial \mathbf{z}^{(L-1)}}{\partial \mathbf{W}^{(L-1)}} \cdot \frac{\partial \mathbf{a}^{(L)}}{\partial \mathbf{z}^{(L-1)}} \cdot \frac{\partial \mathbf{z}^{(L)}}{\partial \mathbf{a}^{(L)}} \cdot \frac{\partial \mathbf{a}^{(L+1)}}{\partial \mathbf{z}^{(L)}} \cdot \frac{\partial J(\cdot)}{\partial \mathbf{a}^{(L+1)}}$$

(4b) (4) (3) (2) (1)

keep chaining

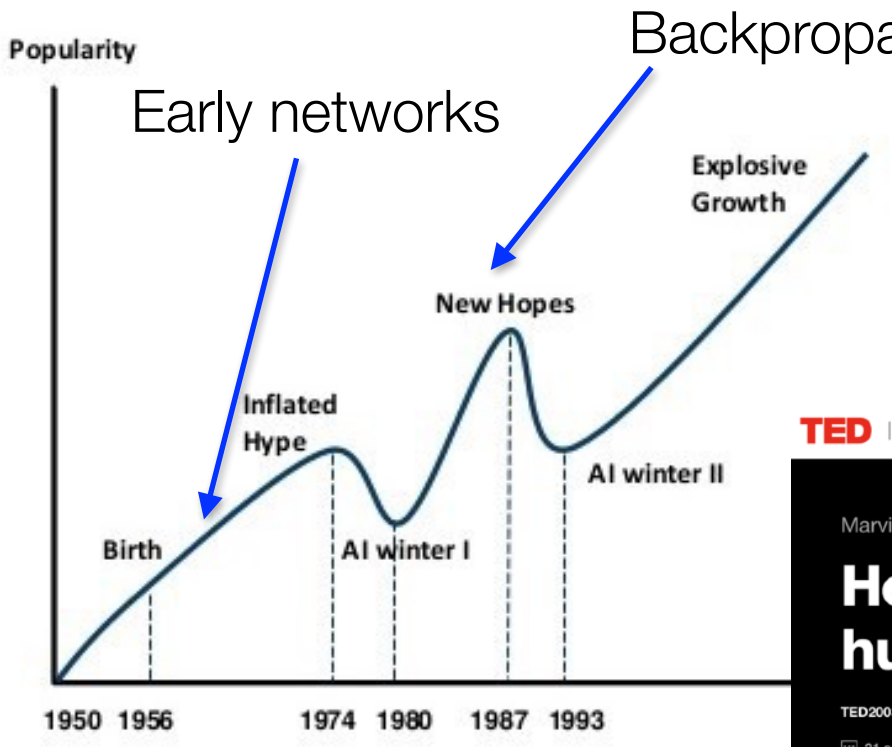
Back propagation



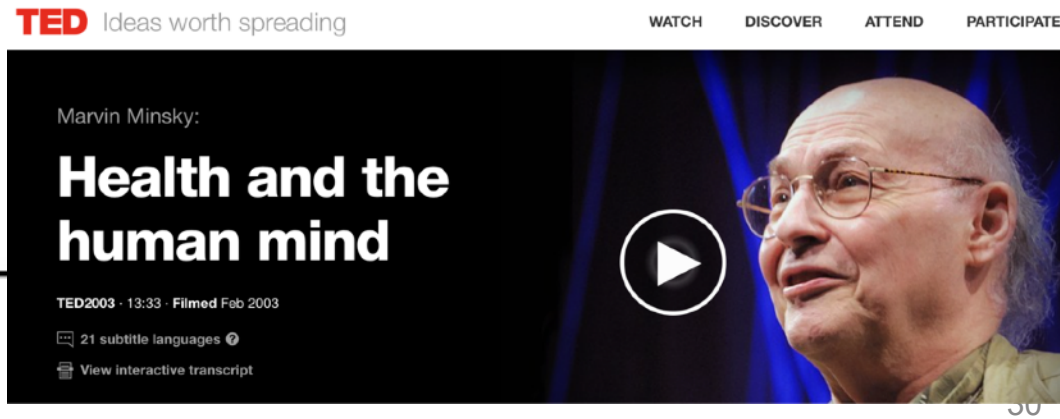


You are ready to begin back propagation!

The First AI Winter (if time)







The Rosenblatt-Widrow-Hoff Dilemma

- 1960's: Rosenblatt got into a public academic argument with Marvin Minsky and Seymour Papert

> "Given an elementary α-perceptron, a stimulus world W, and any classification C(W) for which a solution exists; let all stimuli in W occur in any sequence, provided that each stimulus must reoccur in finite time; then beginning from an arbitrary initial state, an error correction procedure will always yield a solution to C(W) in finite time…"

- Minsky and Papert publish limitations paper, 1969:

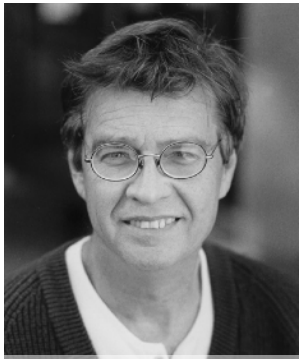
> "the style of research being done on the perceptron is doomed to failure because of these limitations."

- Widrow and Rosenblatt try to build bigger networks without limitations and fail
  - ⋆ Neural Networks research **basically stops** for **17 years**
- **Until**: researchers revisit training bigger networks
  - ⋆ Neural Networks with multiple layers

Stable Training of Multi-layer Architectures: history

- 1986: *Rumelhart*, *Hinton*, and *Williams* popularize gradient calculation for multi-layer network
  - *technically* introduced by Werbos in 1982
- **difference**: Rumelhart *et al.* validated ideas with a computer
- until this point no one could train a multiple layer network consistently
- algorithm is popularly called **Back-Propagation**
- wins pattern recognition prize in 1993, becomes de-facto machine learning algorithm until: SVMs and Random Forests in ~2004
- would eventually see a resurgence for its ability to train algorithms for Deep Learning applications: **Hinton is widely considered the founder of deep learning**

David Rumelhart



1942-2011

Geoffrey Hinton



End of Session

- thanks! **Next time is Flipped Assignment!!!**

More help on neural networks to prepare for next time:

Dr. Sebastian Raschka
https://github.com/rasbt/python-machine-learning-book/blob/master/code/ch12/ch12.ipynb

Dr. Martin Hagan
https://www.google.com/url?sa=t&rct=j&q=&esrc=s&source=web&cd=1&cad=rja&uact=8&ved=0ahUKEwioprvn27fPAhWMx4MKHYbwDIwQFggeMAA&url=http%3A%2F%2Fhagan.okstate.edu%2FNNDesign.pdf&usg=AFQjCNG5YbM4xSMm6K5HNsG-4Q8TvOu_Lw&sig2=bgT3k-5ZDDTPZ07Qu8Oreg

Dr. Michael Nielsen
http://neuralnetworksanddeeplearning.com